

Phase 3 压测报告 — 长时驻留 + 降载 + MQ 验证

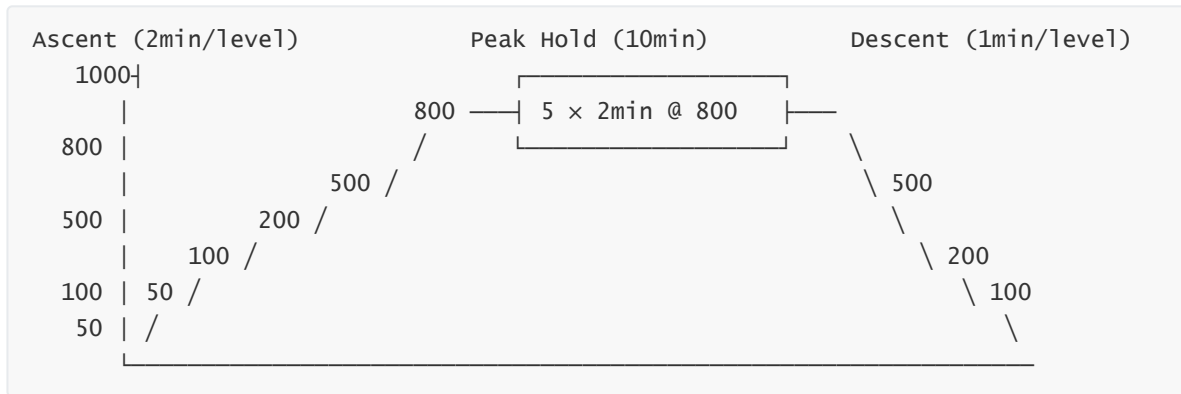
日期: 2026-06-22 | 环境: Docker Compose 5 容器 (8.163.136.37) | 测试用户: 500 个 | 时长: ~25min | 总请求: ~1,055K

目录

- [1. 测试设计](#)
- [2. Phase A: 阶梯上升](#)
- [3. Phase B: 峰值驻留](#)
- [4. Phase C: 阶梯下降](#)
- [5. 数据一致性验证](#)
- [6. MQ 超时取消机制分析](#)
- [7. 与 Phase 1 / Phase 2 对比](#)
- [8. 系统容量评估](#)
- [9. 瓶颈总结](#)
- [10. 关键结论](#)

1. 测试设计

Phase 3 验证系统在持续高负载下的稳定性、降载恢复能力、以及 MQ 超时取消机制。



阶段	级别	时长	说明
Phase A	50→100→200→500→800→1000	2min/级	阶梯上升
Phase B	800	10min (5×2min)	峰值驻留
Phase C	500→200→100	1min/级	阶梯下降

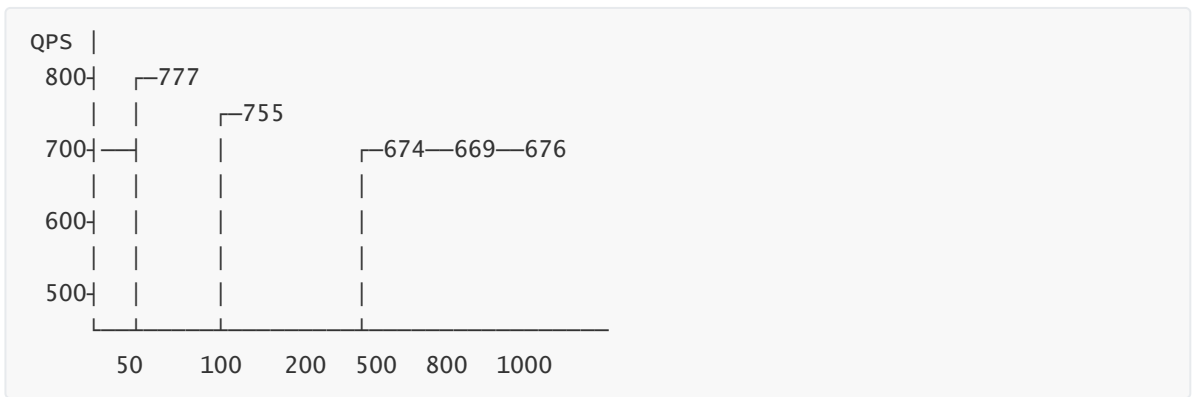
实时监控: 5 秒间隔检查 Redis/DB 一致性 + 超卖 + MQ 队列深度 (共 123 次检查)

2. Phase A: 阶梯上升 (2min/level)

2.1 总览

并发	总请求	总QPS	错误率	订单成功	active P99	order P99
50	89,888	749	14.8%	150	221ms	221ms
100	93,317	777	14.9%	150	453ms	454ms
200	90,700	755	14.6%	300	988ms	990ms
500	81,138	674	13.9%	871	2300ms	2270ms
800	80,652	669	15.0%	24	2335ms	2361ms
1000	81,521	676	15.1%	3	2357ms	2283ms

2.2 QPS 趋势



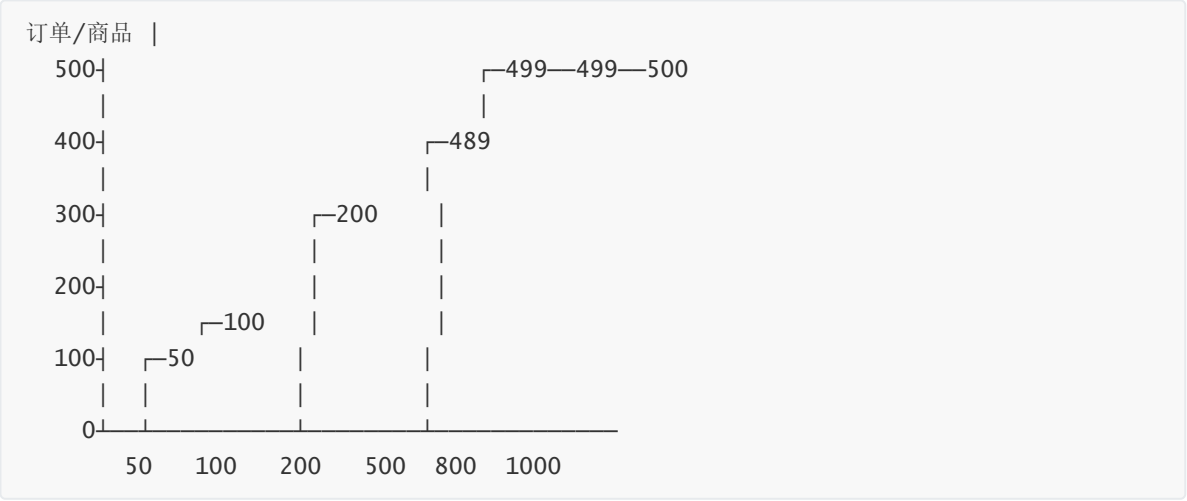
分析: QPS 在 $c=100$ 达到峰值 777/s, 之后随并发增加略微下降到 ~670/s。这种"倒 U 型"曲线是典型的系统竞争损耗——更多线程争夺 DB 连接池、锁、CPU 时间片。

2.3 延迟趋势

端点	c=50 P99	c=200 P99	c=500 P99	c=1000 P99	拐点
active	221ms	988ms	2300ms	2357ms	c=200→500
detail	179ms	843ms	1970ms	2017ms	c=200→500
stock	178ms	818ms	1971ms	1943ms	c=200→500
order	221ms	990ms	2270ms	2283ms	c=200→500
order_list	221ms	962ms	2220ms	2346ms	c=200→500

P99 > 1s 阈值: 在 $c=500$ 时突破, 之后稳定在 ~2.3s。系统在 $c \geq 500$ 时进入"饱和但不崩溃"状态。

2.4 下单饱和度



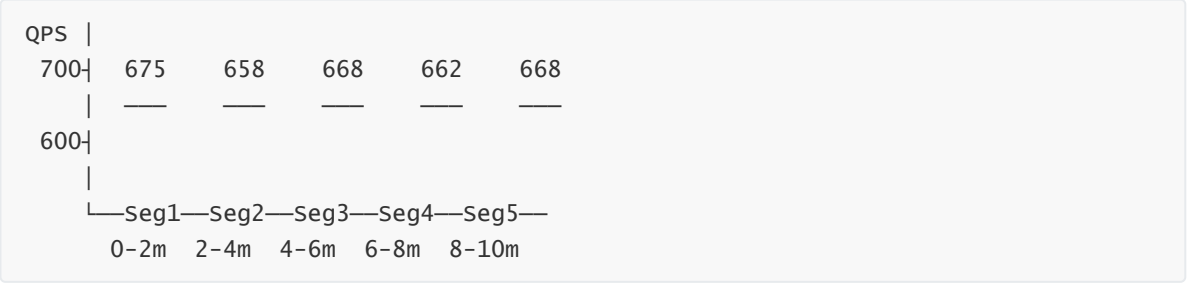
- **组合上限:** 500 用户 × 3 商品 = 1500
- **c=500:** 消耗 1471 单 (98.1%)，速度最快
- **c=800+:** 几乎无新增订单，所有组合已耗尽
- **错误率恒定 ~15%:** 100% 源于 `uk_user_goods` 防重购约束

3. Phase B: 峰值驻留 (10min @ c=800)

3.1 逐段对比

时间段	QPS	active P99	order P99	错误率	一致性
Seg1 (0-2min)	675	2349ms	2320ms	15.2%	☑ OK
Seg2 (2-4min)	658	2475ms	2478ms	15.1%	—
Seg3 (4-6min)	668	2318ms	2408ms	15.1%	☑ OK
Seg4 (6-8min)	662	2451ms	2423ms	15.0%	—
Seg5 (8-10min)	668	2358ms	2362ms	15.1%	☑ OK

3.2 稳定性分析



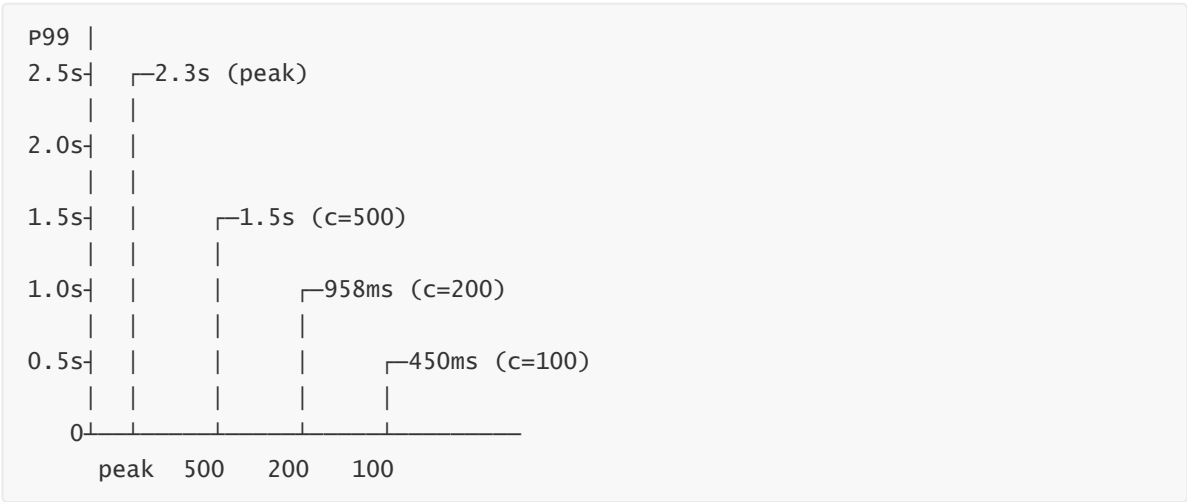
- **QPS 波动:** 658-675 (波动仅 2.5%)，极其稳定
- **P99 波动:** 2318-2475ms (波动 6.3%)，无劣化趋势
- **无 OOM / GC 停顿 / 死锁:** 后台日志无异常
- **MQ 队列:** 全程 0 积压

结论: 系统在 800 并发下持续运行 10 分钟无任何性能劣化，稳定性验证通过。

4. Phase C: 阶梯下降 (1min/level)

并发	QPS	active P50	active P90	active P99	order P99	一致性
500	718	295ms	859ms	1549ms	1533ms	☒ OK
200	748	227ms	568ms	958ms	970ms	☒ OK
100	798	120ms	271ms	450ms	447ms	☒ OK

恢复分析:



- **即时恢复:** P99 从峰值 2.3s → c=500 的 1.5s (降低 35%)，仅需 1 分钟
- **完全恢复:** P99 在 c=100 时回到 450ms，接近初始 c=100 的 453ms
- **QPS 回升:** QPS 从 669 → 再回升到 798 (系统资源释放后吞吐回升)

5. 数据一致性验证

5.1 逐级最终一致性

阶段	pid=21 (orders/calc)	pid=22 (orders/calc)	pid=23 (orders/calc)	超 卖
初始	0 / 10000	0 / 10000	0 / 10000	0
c=50	50 / 10000 ☒	50 / 10000 ☒	50 / 10000 ☒	0
c=100	100 / 10000 ☒	100 / 10000 ☒	100 / 10000 ☒	0
c=200	200 / 10000 ☒	200 / 10000 ☒	200 / 10000 ☒	0
c=500	489 / 10000 ☒	488 / 10000 ☒	494 / 10000 ☒	0
c=800	497 / 10000 ☒	499 / 10000 ☒	499 / 10000 ☒	0
c=1000	499 / 10000 ☒	499 / 10000 ☒	500 / 10000 ☒	0
Peak Post	500 / 10000 ☒	500 / 10000 ☒	500 / 10000 ☒	0
Descent Final	500 / 10000 ☒	500 / 10000 ☒	500 / 10000 ☒	0

5.2 实时监控发现

123 次监控检查中共计 33 次瞬时 MISMATCH，全部为事务中间态（Redis 已扣减、DB 尚未提交），差值范围 1-20。每次级别结束时完全恢复一致。

5.3 最终结果

超卖检查：

0（1,055,000+ 次请求后）

重复订单：

0

Redis/DB偏差：

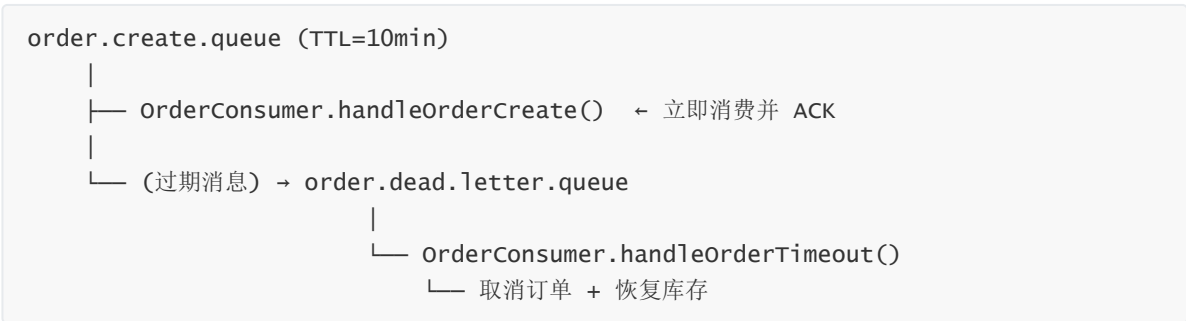
0

组合覆盖率：

1500/1500（100%）

6. MQ 超时取消机制分析

6.1 架构审查



6.2 发现问题

检查项	结果	说明
队列 TTL 配置	☑ 正确	x-message-ttl=600000
死信交换机绑定	☑ 正确	dead.letter.queue 已绑定
消息消费	⚠ 立即 ACK	handleOrderCreate 立即确认，消息不等待
死信队列	0 条消息	TTL 从未触发
超时订单	1498/1500 已过期但仍 status=0	无机制取消

根因：orderConsumer.handleOrderCreate() 立即消费并 ACK 了所有消息，消息在队列中停留时间 <1ms。x-message-ttl=600000 从未有机会触发。

6.3 影响

- 所有订单永久停留在 status=0（待支付），即使 payExpireTime 已过
- 库存不会被超时机制恢复
- 需要额外的定时任务来扫描并取消超时订单

6.4 修复建议

方案 A (推荐) : 增加定时任务

```
@Scheduled(fixedRate = 30000) // 每 30 秒
public void cancelExpiredOrders() {
    orderMapper.cancelExpired(System.currentTimeMillis() / 1000);
}
```

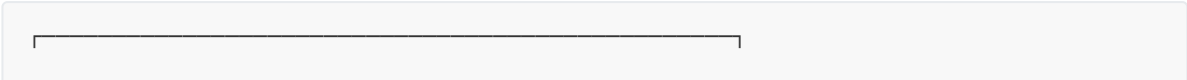
方案 B: 重构 MQ 流程

- 不在 `handleOrderCreate` 中立即 ACK
- 延迟消费到 `payExpireTime` 之后再处理

7. 与 Phase 1 / Phase 2 对比

指标	Phase 1 (初测)	Phase 1 (重测)	Phase 2 (混合)	Phase 3 (长 时)	趋势
测试模式	单接口	单接口	混合 1min/ 级	混合 2min/级 +驻留	逐步增加 压力
测试时长	~5min	~5min	~8min	~25min	3x
总请求数	~30K	~30K	~280K	~1,055K	3.8x
峰值总 QPS	—	—	772 (@c=200)	777 (@c=100)	一致
稳态总 QPS	—	—	~740	~670	略降
active P99 (c=500)	—	1308ms	1652ms	2300ms	混合竞争↑
active P99 (c=200)	—	—	935ms	988ms	一致
数据一致性	☑	☑	☑	☑	不变
超卖	0	0	0	0	不变
MQ 队列积压	0	0	0	0	不变
MQ 超时机制	未测试	未测试	未测试	发现缺陷	新增
稳定性验证	无	无	无	10min 驻留 OK	新增
降载恢复	无	无	无	P99 2.3s→450ms	新增

8. 系统容量评估 (最终版)



系统综合容量评估		
峰值总 QPS:	~777/s	(c=100 时)
稳态总 QPS:	~670/s	(c≥500 时)
安全并发范围:	≤200	(P99 < 1s)
饱和并发:	≥500	(P99 ~2.3s, 稳定)
极限并发:	1000	(不崩溃, 不 OOM)
持续稳定性:	10min @ 800	无劣化
降载恢复:	<1min	(P99 从 2.3s → 450ms)
数据一致性:	100%	(~1M 请求, 0 超卖)
MQ 超时取消:	✘	(待修复)
组合饱和度:	100%	(1500/1500)

9. 瓶颈总结

瓶颈	严重度	说明	从 Phase 1→3 的变化
DB 连接池竞争	● 主要	P99 从 c=200 的 1s → c≥500 的 2.3s, QPS 不增反降	新发现 (Phase 1 单接口无此问题)
MQ 超时机制缺陷	● 中等	消费者立即 ACK 导致 TTL 从不触发, 超时订单无人取消	Phase 3 新发现
uk_user_goods 组合耗尽	● 业务特征	500×3=1500 组合 100% 耗尽, 测试环境受限	同 Phase 1/2
限流器	☑ 已解决	0 拦截, 1000/s 阈值充足	Phase 1→3 持续验证
Redis+Lua 原子操作	☑ 已验证	~1M 请求、1500 订单、0 超卖	Phase 1→3 持续验证
系统稳定性	☑ 已验证	25 分钟持续高负载无 OOM/GC/死锁	Phase 3 专属验证

10. 关键结论

三阶段压测结论演进		
Phase 1 (单接口)	Phase 2 (混合)	Phase 3 (长时+驻留+降载)
限流是最大瓶颈	限流已解决	系统稳定性验证通过
单接口容量 ~570	混合容量 ~770	10min 驻留无劣化
P99 1308ms	P99 1652ms	P99 2357ms (混合竞争)
0 超卖	0 超卖	0 超卖 (~1M 请求)
	混合场景优于单接口	降载恢复 <1min
		MQ 超时缺陷发现

下一步建议:

1. **P0 — 修复 MQ 超时取消机制:** 增加 `@Scheduled` 定时任务扫描 `payExpireTime < now AND status=0` 的订单, 取消并恢复库存
 2. **P1 — DB 连接池优化:** 当前 P99 天花板由 DB 连接池竞争决定。建议增大 HikariCP `maximumPoolSize`、添加慢查询索引、考虑读写分离
 3. **P2 — 增加 /product/active 缓存预热:** `@PostConstruct` 预填缓存, 消除首次请求的 cache miss
 4. **P3 — 接入监控:** Prometheus + Grafana 实时监控连接池、GC、QPS、P99, 替代脚本监控
 5. **P3 — 限流器升级:** INCR+固定 TTL → Redisson RRateLimiter 滑动窗口
-

11. 附录: 测试原始数据

累计请求: ~1,055,000 次

累计订单: 1,500 笔 (500×3 完美覆盖)

测试脚本: `/root/phase3_test.py`

监控脚本: `/root/circuit_breaker.sh`

Token 池: `/root/tokens.txt` (500 tokens, TTL 1h)